



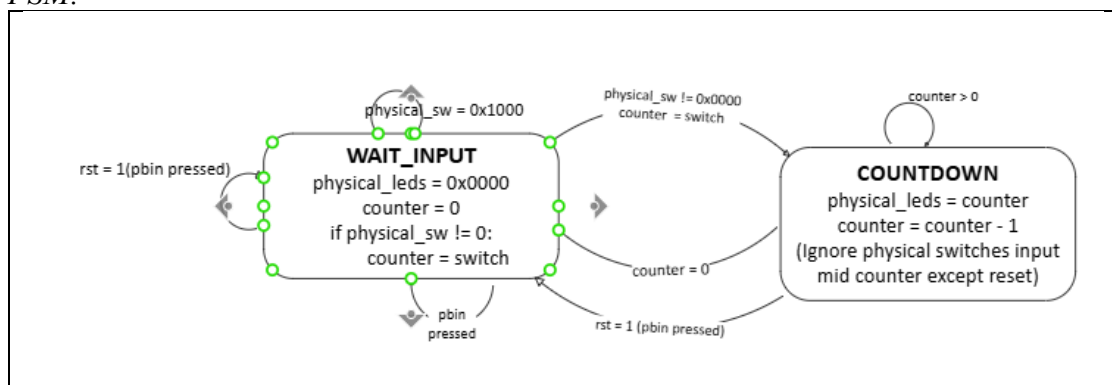
Lab 05 – Worksheet

Name: Abbas Huzaifa Dar, Muhammad Kabir	ID: ad11013, sk11236	Section: T4
---	----------------------	-------------

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

FSM:



Comments

- **Initial Transition and Idle State (Wait for Non-Zero Switch Input):** The diagram correctly includes an arrow leading directly into STATE 0: INPUT_WAITING (Idle),". The "Condition: physical_sw == 0x0000" self-loop on State 0 is that the system remains in the waiting state if all input switches are zero.
- **Capturing and Latching Non-Zero Input:** The transition from State 0 to "STATE 1: COUNTDOWN" is triggered by physical_sw != 0x0000." Critically, this transition is also explicitly labeled with the action "Capture physical_sw into counter."
- **Synchronous Decrement and Input Disabling:** While in the Active/Countdown state (State 1), the FSM relies on a self-loop with counter>0, described as "Count down value. The state node actions further confirm synchronous logic counter = counter - 1 (every slow clk).
- **Automatic Return and Clearing:** When the decrement counter reaches its minimum value, the automatic return logic is executed at counter == 0, which directs the FSM from State 1 back to State 0, labeled with "Return to Idle." The diagram also details the reset behavior, with dedicated arrows pointing from both states back to State 0. These transitions are explicitly noted for their required clearing actions: "Immediate return to State 0, counter=0, LEDs=0."



Task 2

Modules + Testbench:

Top Module:

```
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer: Muddassir Ali

// Module Name: top_fsm_system
// Project Name: Counter
// Target Devices: Baasys 3
//
/////////////////////////////////////////////////////////////////

module top_fsm_system (
    input wire clk,
    input wire pbin,
    input wire [15:0] physical_sw,
    output wire [15:0] physical_leds
);

    // DEBOUNCER (Cleans up the physical reset button signal)
    wire rst_clean;
    wire [31:0] switch_data; // hold the value read from the switches
    reg [31:0] led_write_data = 32'd0; // counter value here
    wire slow_clk;

    debouncer rst_db (
        .clk(clk),
        .pbin(pbin),
        .pbout(rst_clean)    //generated a clean signal
    );

    leds switch_reader (
        .clk(clk), .rst(rst_clean),
        .btns(16'd0),           // Not used for this FSM
        .writeData(32'd0),      // We don't write to switches
        .writeEnable(1'b0),     // Disabled
        .readEnable(1'b1),      // Always ON so we can monitor switches
        .memAddress(30'd0),
        .switches(physical_sw), // Plug in the physical switches
        .readData(switch_data)  // output data
    );

    switches led_writer (
        .clk(clk), .rst(rst_clean),
        .writeData(led_write_data),
        .writeEnable(1'b1),      // Always ON so LEDs update instantly
        .readEnable(1'b0), .memAddress(30'd0),
        .readData(),            // Ignored
    );
```



```
        .leds(physical_leds)
    );

    clock_divider ticker (
        .clk_in(clk),                // Feed it the 100MHz fast clock
        .rst(rst_clean),            // Feed it the clean reset signal
        .clk_out(slow_clk)           // It spits out the 1Hz slow clock!
    );

    // YOUR FSM AND COUNTER LOGIC
    // YOUR CODE HERE
    localparam WAIT_INPUT = 1'b0;
    localparam COUNTDOWN = 1'b1;

    reg state = WAIT_INPUT;

    always @(posedge slow_clk or posedge rst_clean) begin
        if(rst_clean) begin
            // Button reset: immediately return to waiting state and clear counter
            state      <= WAIT_INPUT;
            led_write_data <= 32'd0;
        end
        else begin
            case (state)
                WAIT_INPUT: begin
                    if (switch_data != 32'd0) begin
                        led_write_data <= switch_data;
                        state      <= COUNTDOWN;
                    end
                    else begin
                        led_write_data <= 32'd0;
                        state      <= WAIT_INPUT;
                    end
                end
            end

            COUNTDOWN: begin
                if (led_write_data == 32'd0) begin
                    state      <= WAIT_INPUT;
                    led_write_data <= 32'd0;
                end
                else begin
                    led_write_data <= led_write_data - 32'd1;
                    state      <= COUNTDOWN;
                end
            end

            default: begin
                state      <= WAIT_INPUT;
                led_write_data <= 32'd0;
            end
        endcase
    end
```



```
end  
endmodule
```

Testbench

```
`timescale 1ns / 1ps
```

```
module tb_top_fsm;
```

```
    reg clk;  
    reg pbin;  
    reg [15:0] physical_sw;  
    wire [15:0] physical_leds;
```

```
    // Instantiate the Top Module  
    top_fsm_system top_level (  
        .clk(clk),  
        .pbin(pbin),  
        .physical_sw(physical_sw),  
        .physical_leds(physical_leds)  
    );
```

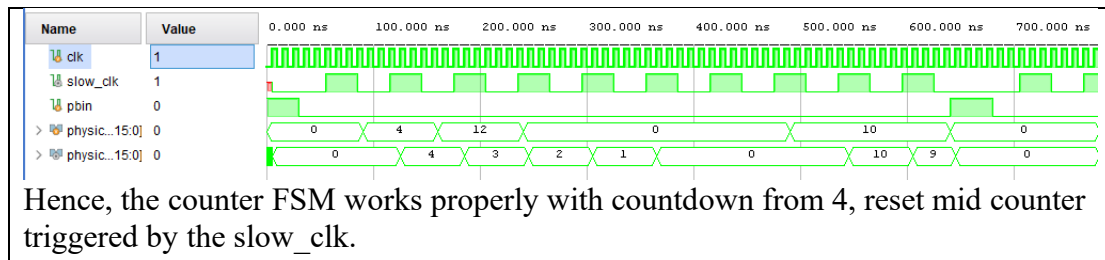
```
    // 100 MHz clock generation (10 ns period)  
    always #5 clk = ~clk;
```

```
    initial begin  
        // Initialize inputs  
        clk = 0;  
        pbin = 1;      // Assert reset initially  
        physical_sw = 16'd0;  
        #30;  
        pbin = 0;  
        #60;
```

```
        // Test 1: Load switch value = 4 and verify countdown to 0  
        physical_sw = 16'd4;  
        #70;  
        // Change switches during countdown to verify switch inputs are ignored  
        physical_sw = 16'd12;  
        #80;  
        physical_sw = 16'd0; // Set switches to 0 before countdown finishes  
        #250;
```

```
        // Test 2: Load switch value = 10 and press reset mid-countdown  
        physical_sw = 16'd10;  
        #150;  
        physical_sw = 16'd0;  
        pbin = 1;      // Press reset button mid-countdown  
        #40;  
        pbin = 0;  
        #100;  
        $finish;
```

```
    end  
endmodule
```

**Results:****Task 3***XDC file*

```
# Clock (100 MHz)
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]

# Reset button (center button, BTNC)
set_property PACKAGE_PIN U18 [get_ports pbin]
set_property IOSTANDARD LVCMOS33 [get_ports pbin]

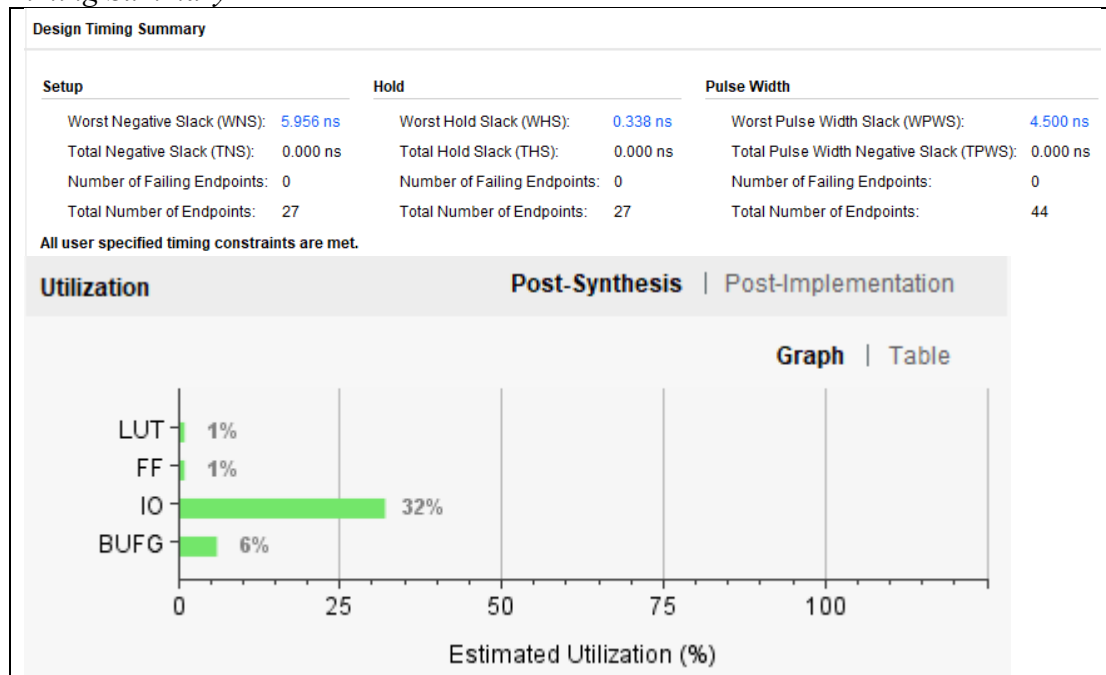
# Switches
set_property PACKAGE_PIN V17 [get_ports {physical_sw[0]}]
set_property PACKAGE_PIN V16 [get_ports {physical_sw[1]}]
set_property PACKAGE_PIN W16 [get_ports {physical_sw[2]}]
set_property PACKAGE_PIN W17 [get_ports {physical_sw[3]}]
set_property PACKAGE_PIN W15 [get_ports {physical_sw[4]}]
set_property PACKAGE_PIN V15 [get_ports {physical_sw[5]}]
set_property PACKAGE_PIN W14 [get_ports {physical_sw[6]}]
set_property PACKAGE_PIN W13 [get_ports {physical_sw[7]}]
set_property PACKAGE_PIN V2 [get_ports {physical_sw[8]}]
set_property PACKAGE_PIN T3 [get_ports {physical_sw[9]}]
set_property PACKAGE_PIN T2 [get_ports {physical_sw[10]}]
set_property PACKAGE_PIN R3 [get_ports {physical_sw[11]}]
set_property PACKAGE_PIN W2 [get_ports {physical_sw[12]}]
set_property PACKAGE_PIN U1 [get_ports {physical_sw[13]}]
set_property PACKAGE_PIN T1 [get_ports {physical_sw[14]}]
set_property PACKAGE_PIN R2 [get_ports {physical_sw[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[*]}]

# LEDs
set_property PACKAGE_PIN U16 [get_ports {physical_leds[0]}]
set_property PACKAGE_PIN E19 [get_ports {physical_leds[1]}]
set_property PACKAGE_PIN U19 [get_ports {physical_leds[2]}]
set_property PACKAGE_PIN V19 [get_ports {physical_leds[3]}]
set_property PACKAGE_PIN W18 [get_ports {physical_leds[4]}]
set_property PACKAGE_PIN U15 [get_ports {physical_leds[5]}]
set_property PACKAGE_PIN U14 [get_ports {physical_leds[6]}]
set_property PACKAGE_PIN V14 [get_ports {physical_leds[7]}]
```



```
set_property PACKAGE_PIN V13 [get_ports {physical_leds[8]}]
set_property PACKAGE_PIN V3 [get_ports {physical_leds[9]}]
set_property PACKAGE_PIN W3 [get_ports {physical_leds[10]}]
set_property PACKAGE_PIN U3 [get_ports {physical_leds[11]}]
set_property PACKAGE_PIN P3 [get_ports {physical_leds[12]}]
set_property PACKAGE_PIN N3 [get_ports {physical_leds[13]}]
set_property PACKAGE_PIN P1 [get_ports {physical_leds[14]}]
set_property PACKAGE_PIN L1 [get_ports {physical_leds[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[*]}]
```

Timing Summary



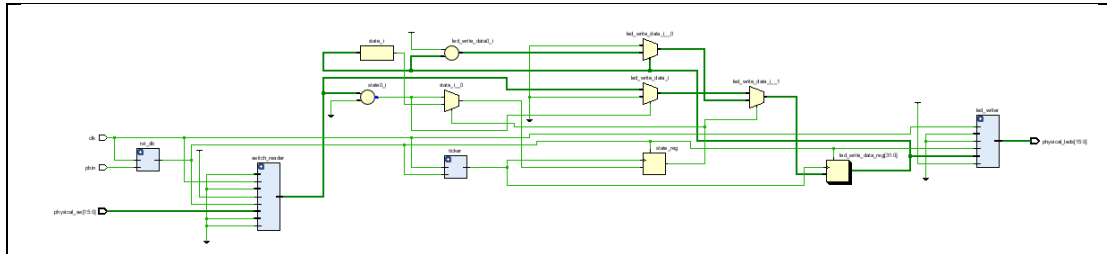
Interpretation:

The Vivado Timing Summary shows that the FSM design successfully meets all the timing constraints because the WNS is strongly positive (+5.956ns) and the Total Negative Slack (TNS) is 0.000ns with 0 failing endpoints across all Setup, Hold (0.338ns WHS), and Pulse Width (+4.500ns WPWS) checks, all signals propagate and stabilize well within the 10.000ns and the scaled up 1s (scaled down frequency) clock period which easily satisfies the timing requirements as the FSM state transitions are driven by that significantly scaled-down clock signal.

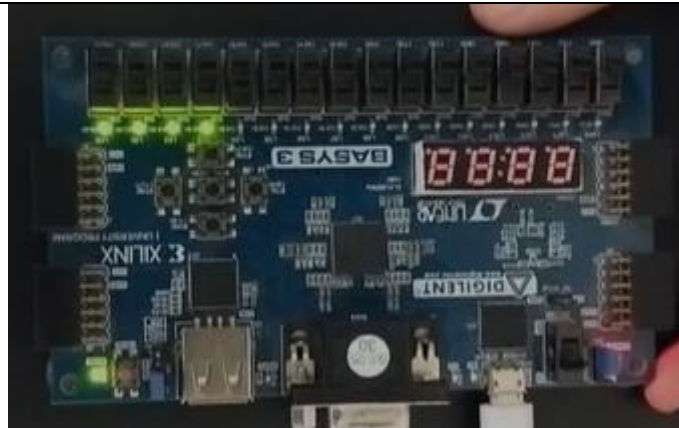


Task 4

RTL Schematic:



Results



Counter FSM video of FPGA shown to RA and available in [Github Repository](#) Lab 05 folder.

Reference:

https://github.com/achdiop/CA_Labs-Git-Abbas



Lab 05 – Designing FSM Using FPGA Switches and LEDs

Assessment Rubrics

Points Distribution

Task No.	LR2 Code	LR 5 Results	LR11 Design	AR7
Prelab		/15		/10 & /10
Task 1	-	-	/10	
Task 2	/15	/10		
Task 3		/15		
Task 4		/15		
Total Points	/80			/20
CLO Mapped	CLO 1			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures	Good summary of most the in-lab tasks is included in report. The work is supported by figures and	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly



			/ plots are not included. Report is submitted after due date.	plots with explanations. The report is submitted timely.	including all the necessary figures, plots and tables.
--	--	--	---	---	---